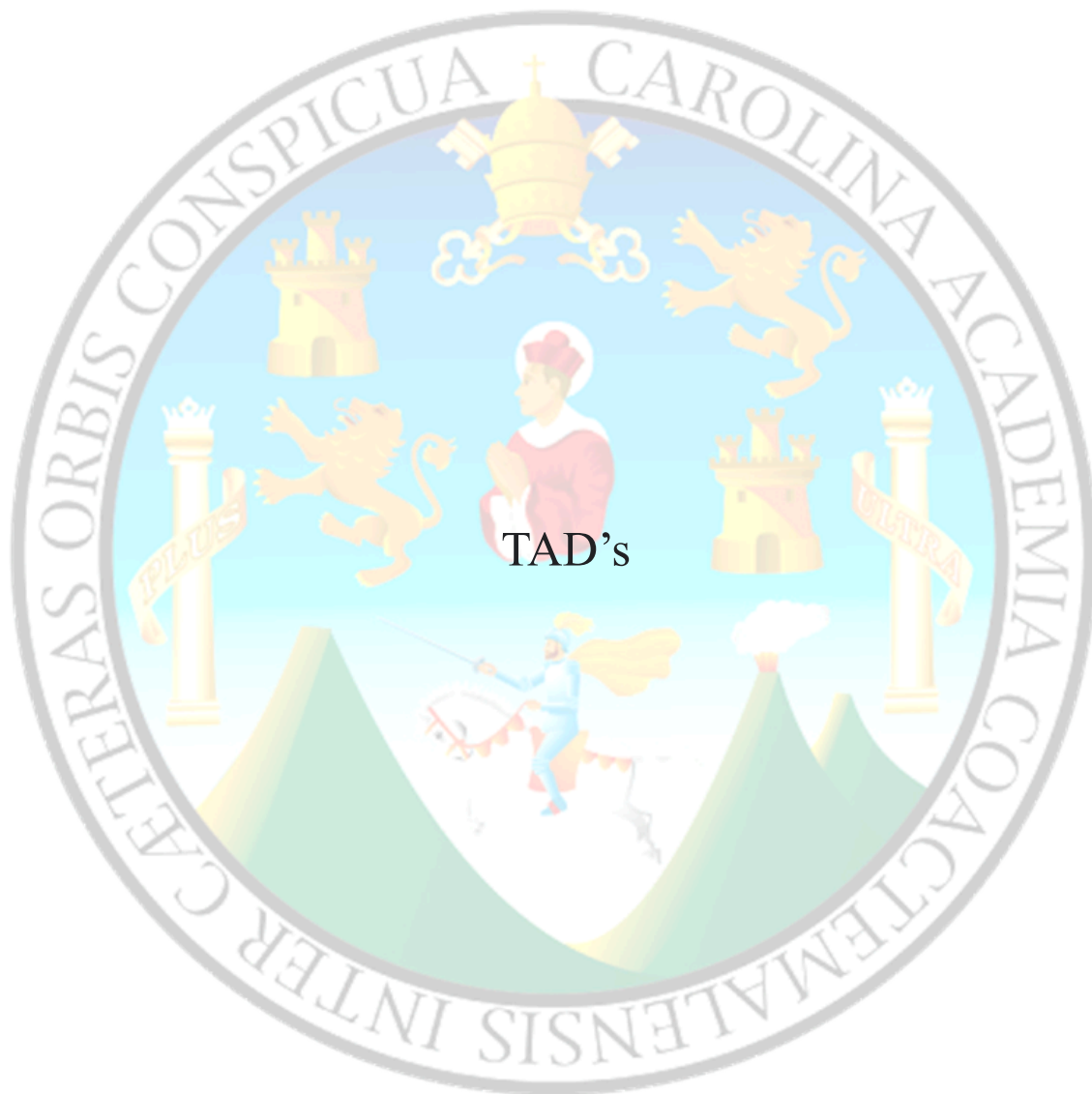


UNIVERSIDAD SAN CARLOS DE GUATEMALA
CENTRO UNIVERSITARIO DE OCCIDENTE
DIVISIÓN DE CIENCIAS DE LA INGENIERÍA
ESTRUCTURA DE DATOS



Mauricio Joel Gómez Barrios - 202031478

24 de febrero de 2026

Tipos Abstractos de Datos y sus Estructuras.

A continuación se detallan los TADs implementados.

1) TAD Nodo (Template):

Estructura base para la construcción de todas los arreglos dinámicos del proyecto.

```
template <class T>
class Nodo
{
public:
    T dato; // Plantilla genérica que almacena el valor (puede ser Carta* o Jugador*).
    Nodo<T>* siguiente; // Puntero al siguiente elemento.
    Nodo<T>* anterior; // Puntero al elemento previo, para listas dobles y circulares
    Nodo(T valor)
    {
        this->dato = valor;
        this->siguiente = nullptr;
        this->anterior = nullptr;
    }
};
```

2) TAD Pila (Template):

Estructura de tipo último en entrar, primero en salir, utilizada para gestionar el mazo principal y el mazo de descarte.

```
#ifndef PILA_H
#define PILA_H

#include <iostream>
#include <cstdlib>
#include <ctime>
#include "Nodo.h"
#include "Carta.h"

using namespace std;

template <class T>
class Pila
{
private:
    Nodo<T>* tope; // Puntero a la carta que está en la parte superior.
    int tamaño; // Contador de elementos.

public:
    Pila()
    {
        tope = nullptr;
        tamaño = 0;
    }
};
```

```

        srand(time(0));
    }

~Pila()
{
    while (!estaVacia())
    {
        desapilar();
    }
}

bool estaVacia()
{
    return tope == nullptr;
}

int getTamaño()
{
    return tamaño;
}

void apilar(T dato) // Inserta un elemento en el tope.
{
    Nodo<T>* nuevo = new Nodo<T>(dato);
    nuevo->siguiente = tope;
    tope = nuevo;
    tamaño++;
}

T desapilar() // Retira y devuelve el elemento del tope.
{
    if (estaVacia())
    {
        return nullptr;
    }
    Nodo<T>* temp = tope;
    T dato = temp->dato;
    tope = tope->siguiente;
    delete temp;
    tamaño--;
    return dato;
}

T verTope() // Devuelve el elemento del tope sin sacarlo.
{
    if (estaVacia()) return nullptr;
    return tope->dato;
}

void barajar() // Mezcla los nodos.

```

```

{
    if (tamaño < 2) return;
    T* arr = new T[tamaño];
    Nodo<T>* actual = tope;
    int count = 0;
    while(actual != nullptr)
    {
        arr[count++] = actual->dato;
        actual = actual->siguiente;
    }
    for (int i = tamaño - 1; i > 0; i--)
    {
        int j = rand() % (i + 1);
        T temp = arr[i];
        arr[i] = arr[j];
        arr[j] = temp;
    }
    actual = tope;
    count = 0;
    while(actual != nullptr)
    {
        actual->dato = arr[count++];
        actual = actual->siguiente;
    }
    delete[] arr;
}

```

void eliminarPorTipo(TipoCarta tipoObjetivo) // Elimina cartas específicas de la pila.

```

{
    if (estaVacia()) return;
    while (tope != nullptr && tope->dato->getTipo() == tipoObjetivo)
    {
        Nodo<T>* aBorrar = tope;
        tope = tope->siguiente;
        delete aBorrar->dato;
        delete aBorrar;
        tamaño--;
    }
    if (tope == nullptr) return;
    Nodo<T>* actual = tope;
    while (actual->siguiente != nullptr)
    {
        if (actual->siguiente->dato->getTipo() == tipoObjetivo)
        {
            Nodo<T>* aBorrar = actual->siguiente;
            actual->siguiente = aBorrar->siguiente;
            delete aBorrar->dato;
            delete aBorrar;
            tamaño--;
        }
    }
}

```

```

        }
        else
        {
            actual = actual->siguiente;
        }
    }
}
};
#endif

```

3) TAD ListaDoble (Template):

Estructura en secuencia con navegación en ambas direcciones, utilizada para representar la mano del jugador.

```

#ifndef LISTADOBLE_H
#define LISTADOBLE_H
#include "Nodo.h"
#include "Carta.h"
using namespace std;

template <class T>
class ListaDoble
{
private:
    Nodo<T>* cabeza; // Puntero a la primera carta.
    Nodo<T>* cola; // Puntero a la última carta.
    int tamaño; // Cantidad de cartas en la mano.

public:
    ListaDoble()
    {
        cabeza = nullptr;
        cola = nullptr;
        tamaño = 0;
    }

    int getTamaño()
    {
        return tamaño;
    }

    void insertarAlFinal(T dato) // Agrega una carta a la mano.
    {
        Nodo<T>* nuevo = new Nodo<T>(dato);
        if (cabeza == nullptr)
        {
            cabeza = cola = nuevo;
        }
    }
}

```

```

else
{
    cola->siguiente = nuevo;
    nuevo->anterior = cola;
    cola = nuevo;
}
tamaño++;
}

```

void eliminar(T dato) // Retira una carta exacta de la mano.

```

{
    if (cabeza == nullptr) return;
    Nodo<T>* actual = cabeza;
    while (actual != nullptr)
    {
        if (actual->dato == dato)
        {
            if (actual == cabeza && actual == cola)
            {
                cabeza = cola = nullptr;
            }
            else if (actual == cabeza)
            {
                cabeza = cabeza->siguiente;
                cabeza->anterior = nullptr;
            }
            else if (actual == cola)
            {
                cola = cola->anterior;
                cola->siguiente = nullptr;
            }
            else
            {
                actual->anterior->siguiente = actual->siguiente;
                actual->siguiente->anterior = actual->anterior;
            }
            delete actual;
            tamaño--;
            return;
        }
        actual = actual->siguiente;
    }
}

```

T obtenerPorIndice(int indice) // Permite navegar la lista como si fuera un arreglo.

```

{
    if (indice < 0 || indice >= tamaño) return nullptr;
    Nodo<T>* actual = cabeza;
    for (int i = 0; i < indice; i++)

```

```

        {
            actual = actual->siguiente;
        }
        return actual->dato;
    }

    void eliminarPorTipo(TipoCarta tipoObjetivo) // Destruye todas las cartas de
    un mismo tipo.
    {
        Nodo<T>* actual = cabeza;
        while (actual != nullptr)
        {
            Nodo<T>* siguienteTemp = actual->siguiente;
            if (actual->dato->getTipo() == tipoObjetivo)
            {
                T cartaABorrar = actual->dato;
                delete cartaABorrar;
                eliminar(cartaABorrar);
                cout << " [Mano] Carta eliminada." << endl;
            }
            actual = siguienteTemp;
        }
    }
};

#endif

```

4) TAD ListaCircular (Template):

Estructura de nodos doblemente enlazados en círculo, utilizada para gestionar el control de turnos de los jugadores.

```

#ifndef LISTACIRCULAR_H
#define LISTACIRCULAR_H

#include "Nodo.h"

template <class T>
class ListaCircular
{
private:
    Nodo<T>* actual; // Puntero al jugador que tiene el turno actual.
    int tamaño; // Número de jugadores en la mesa.

public:
    ListaCircular()
    {
        actual = nullptr;
    }

```

```

    tamaño = 0;
}

int getTamaño()
{
    return tamaño;
}

```

void insertar(T dato) // Agrega a un jugador al círculo y conecta sus extremos.

```

{
    Nodo<T>* nuevo = new Nodo<T>(dato);
    if (actual == nullptr)
    {
        actual = nuevo;
        actual->siguiente = actual;
        actual->anterior = actual;
    }
    else
    {
        Nodo<T>* ultimo = actual->anterior;
        ultimo->siguiente = nuevo;
        nuevo->anterior = ultimo;
        nuevo->siguiente = actual;
        actual->anterior = nuevo;
    }
    tamaño++;
}

```

void siguiente() // Pasa el turno al jugador de la derecha.

```

{
    if (actual != nullptr)
    {
        actual = actual->siguiente;
    }
}

```

void anterior() // Pasa el turno al jugador de la izquierda.

```

{
    if (actual != nullptr)
    {
        actual = actual->anterior;
    }
}

```

T obtenerActual() // Retorna al jugador que le toca jugar.

```

{
    if (actual == nullptr) return nullptr;
    return actual->dato;
}

```



```

    }

    void eliminarActual()
    {
        if (actual == nullptr) return;
        Nodo<T>* aBorrar = actual;
        if (actual->siguiente == actual)
        {
            actual = nullptr;
        }
        else
        {
            Nodo<T>* anterior = actual->anterior;
            Nodo<T>* siguiente = actual->siguiente;
            anterior->siguiente = siguiente;
            siguiente->anterior = anterior;
            actual = siguiente;
        }
        delete aBorrar;
        tamaño--;
    }

    Nodo<T>* getNodoActual()
    {
        return actual;
    }
};

#endif

```

5) TAD Carta:

Representa una carta física del juego y su conexión lógica con su otra cara (Modo Flip).

```

#ifndef CARTA_H
#define CARTA_H
#include <string>
#include <iostream>
using namespace std;

enum Color
{
    ROJO, AZUL, VERDE, AMARILLO, NEGRO, // Cara principal
    ROSA, TURQUESA, NARANJA, VIOLETA // Cara flip
};

enum TipoCarta

```

```

    {
        NUMERO, SALTO, MAS_UNO, // Cartas comodin simples
        COMODIN_COLOR, MAS_DOS, MAS_CUATRO, FLIP, // Comodines cara
principal
        SALTO_TODOS, MAS_TRES, MAS_SEIS, COLOR_ETERNO, // Cartas comodin
cara flip
        REVERSA, // Carta comodin para ambas caras
        DESTRUCTORA, PASAR_EXTREMO // Mis cartas personalizadas
    };

class Carta
{
    private:
        Color color;
        TipoCarta tipo;
        int valor; // 0-9 para cartas numéricas, -1 para especiales
        Carta* ladoOscuro; // Puntero al reverso flip
        bool esLadoOscuro;
    public:
        Carta(Color color, TipoCarta tipo, int valor);
        Color getColor();
        TipoCarta getTipo();
        int getValor();
        Carta* getLadoOscuro();
        bool getEsLadoOscuro();
        void setLadoOscuro(Carta* reverso);
        void setColor(Color nuevoColor);
        void voltear(); // Cambia la carta al lado opuesto visualmente
        bool esCompatible(Carta* otra); // Valida si se puede jugar sobre otra
        string toString();
};

#endif

```

6) TAD Jugador:

Representa a una persona sentada en la mesa con sus cartas.

```

#ifndef JUGADOR_H
#define JUGADOR_H
#include <string>
#include "ListaDoble.h"
#include "Carta.h"
using namespace std;

class Jugador
{
    private:

```

```

    string nombre; // Identificador del jugador.
    ListaDoble<Carta*>* mano; // Instancia del TAD ListaDoble que contiene sus
    cartas.
    public:
        Jugador(string nombre);
        ~Jugador();
        string getNombre();
        ListaDoble<Carta*>* getMano();
        void robarCarta(Carta* c); // Agrega cartas a su estructura.
        Carta* jugarCarta(int indice); // Toma y descarta una carta.
        bool tieneCarta(TipoCarta tipo);
        void aplicarDestruccion(TipoCarta tipo); // Recorre su mano para eliminar
        cartas específicas (poder de la Destructora).
        void ordenarMano(bool ladoFlipActivo); // Agrupa sus cartas por color y
        numeracion.
        int aplicarDestruccionExacta(Color col, TipoCarta tip, int val);
        int cantidadCartas();
};

#endif

```

7) TAD Juego:

Clase principal que integra todos los TADs anteriores y procesa el flujo del juego.

```

#ifndef JUEGO_H
#define JUEGO_H
#include "Pila.h"
#include "ListaCircular.h"
#include "Jugador.h"
#include "Carta.h"

struct ReglasFlags
{
    bool acumulacion = false;
    bool retoMasCuatro = false;
    bool roboHastaJugar = false;
    bool gritoUno = false;
    bool ganarConNegra = false;
    bool modoFlip = false;
};

class Juego
{
    private:
        Pila<Carta*>* mazo; // Lugar central de robo.
        Pila<Carta*>* descarte; // Pila central de juego.
        ListaCircular<Jugador*>* jugadores; // Círculo de turnos.

```

```

ReglasFlags reglas; // Configuracion booleana de reglas
bool direccionDerecha;
bool ladoFlipActivo;
int cartasAcumuladas;
void generarCartas(int numMazos);
void repartirCartasIniciales();
void limpiarPantalla();
void reponerMazo(); // Mueve el descarte de vuelta al mazo principal.
void aplicarCartaEspecial(Carta* cartaJugada, bool teniaOpcionDeCarta);
// Aplica las cartas especiales y personalizadas
void mostrarMesa(Jugador* actual, Carta* cartaTopeVisual);
TipoCarta tipoAcumulado;
Color pedirColorUsuario();
public:
    Juego();
    ~Juego();
    void inicializar(int numJugadores, ReglasFlags configuracion); // Crea los
mazos y crea a los jugadores.
    void flujoPrincipal();
    void gestionarTurno(); // Logica de juego como los menus, robos,
validaciones, etc.
    Jugador* verificarGanador();
    int getCartasEnMazo();
    int getCartasEnDescarte();
    int getCantidadJugadores();
    bool getLadoFlipActivo();
    ListaCircular<Jugador*>* getJugadores();
};

#endif

```